Toxic Tweet Detection Tool

A JavaFX application for content analysis and user graph visualization.

Welcome to the documentation for the Toxic Tweet Detection Tool. This project is a standalone Java desktop application built with JavaFX. Its primary goal is to provide a tool for analyzing tweet data for toxic content, identifying duplicate or similar tweets, and visualizing how toxic content propagates through a user network.

This tool is built using pure Java algorithms without relying on external machine learning libraries or build tools like Maven, making it a lightweight and self-contained solution for algorithmic analysis.

Table of Contents

Overview

The Toxic Tweet Detection tool loads a collection of tweets from a simple `.csv` or `.txt` file. Once loaded, it performs a three-part analysis and presents the findings in a clean, interactive user interface.

Key Features

Toxic Word Detection: Uses the Rabin-Karp string hashing algorithm to efficiently scan tweets for a predefined list of toxic words.

Tweet Similarity Analysis: Employs a hash-based algorithm (Jaccard similarity on character sets) to find and group tweets that are nearly identical.

User Propagation Graphing: Builds a directed graph of users based on retweet (RT) data. It then uses Depth-First Search (DFS) to identify clusters of users who are spreading toxic content.

Interactive UI: A simple JavaFX interface featuring a master table of all tweets, with highlighting for toxic content and a dedicated area for analysis results.

Installation Instructions

This project is designed to run in an IDE like IntelliJ IDEA without Maven. You must have the JavaFX SDK configured manually.

Prerequisites

Java Development Kit (JDK): Version 11 or higher.

JavaFX SDK: A matching version for your JDK (e.g., JavaFX 21 for JDK 21). You can download it from GluonHQ.

Setup Guide (IntelliJ IDEA)

1. **Download the JavaFX SDK:** Download the SDK from GluonHQ and extract the `.zip` file to a permanent location (e.g., `C:\dev\javafx-sdk-21`).

2. **Create a New Java Project:** In IntelliJ, create a new, empty Java project (not a JavaFX project).

3. **Add Source Files:** Copy all `.java`, `.txt`, and `.csv` files from this project into your project's `src` directory (or the root folder for data files).

4 **Configure Project Library:**

- Go to **File > Project Structure... > Libraries**.
- Click the **+** button and select **Java**.
- Navigate to your extracted JavaFX SDK folder and select the `lib` directory.
- Click **OK** to add it as a project library.

5 **Set VM Run Options:**

- Go to **Run > Edit Configurations...**.
- Click **+** and select **Application**.
- Set the **Main class** to `Main` (or `com.example.toxictweet.Main` if you used a package).
- Click **Modify options** and select **Add VM options**.
- In the **VM options** box, add the following, replacing the path with your own:

```
--module-path "C:\dev\javafx-sdk-21\lib" --add-modu
```

- Click **OK** to save.

Compile and Run (Command Line)

1 Set the path to your JavaFX SDK `lib` folder:

```
# Windows
set JFX_PATH="C:\dev\javafx-sdk-21\lib"

# macOS/Linux
export JFX_PATH=/dev/javafx-sdk-21/lib
```

2 Compile all Java files:

```
# Windows
javac --module-path %JFX_PATH% --add-modules javafx.co

# macOS/Linux
javac --module-path $JFX_PATH --add-modules javafx.cor
```

3 Run the main application:

```
# Windows
java --module-path %JFX_PATH% --add-modules javafx.cor

# macOS/Linux
java --module-path $JFX_PATH --add-modules javafx.cont
```

Usage Guide

Once the application is running, usage is straightforward.

1. **Load Tweets:** Click the "Load Tweets (CSV/TXT)" button. A file chooser will appear. Select the `tweets.csv` file or another compatible file.

2. **Run Analysis:** Click the "Run Full Analysis" button. The application will process all loaded tweets.

3. **Review Results:**
   - The main table will update. Toxic tweets will be marked "Yes" and the offending words highlighted in red.
   - The "Analysis Results" text area at the bottom will populate with two reports: a list of similar tweet groups and a list of toxic user clusters.

Code Structure

The project is contained in a flat directory for simplicity. All logic is encapsulated in distinct classes.

```
/ToxicTweetDetection/
├── Main.java              (JavaFX App, UI, Main Logic)
├── Tweet.java             (Data Model for a tweet)
├── User.java              (Data Model for a graph node)
├── RabinKarp.java         (Algorithm: Toxic Detection)
├── HashSimilarity.java    (Algorithm: Similarity)
├── GraphAnalyzer.java     (Algorithm: Graph Analysis)
├── toxic_words.txt        (Config: List of toxic words)
└── tweets.csv             (Data: Sample tweets)
```

Key Components

`Main.java`: The entry point for the JavaFX application. It builds the UI, sets up event handlers (button clicks, file loading), and orchestrates the calls to the various analysis classes.

`Tweet.java`: A simple POJO (Plain Old Java Object) that stores data for a single tweet, including its ID, user, text, and toxicity status.

`User.java`: Represents a node in the user graph. It stores the `userId` and a `Set` of connections (users this user has retweeted).

`RabinKarp.java`: An implementation of the Rabin-Karp algorithm. It is initialized with the list of toxic words and can efficiently `search()` a new string (a tweet) for any matches.

`HashSimilarity.java`: Calculates similarity between tweets. It preprocesses text, converts it into a `Set` of characters, and then uses the Jaccard similarity index to compare two sets.

`GraphAnalyzer.java`: Builds and analyzes the user graph. It takes the `Map` of all users and uses Depth-First Search (DFS) to find all connected components (clusters) that contain at least one toxic tweeter.

Features & Functionality

Toxic Detection (Rabin-Karp)

The `RabinKarp.java` class implements a rolling hash algorithm. It pre-computes the hash value for every word in `toxic_words.txt`. When a tweet is analyzed, it slides a window (the length of a given toxic word) across the tweet text, calculating the hash of the current text segment in O(1) time. If the segment's hash matches a toxic word's hash, it performs a final character-by-character check to confirm the match, avoiding hash collisions.

Similarity Detection (Hash Sets)

The `HashSimilarity.java` class finds near-duplicates. It ignores case and punctuation. For two tweets, it generates a `Set` of unique characters for each. The similarity is then calculated using the Jaccard Index:

```
Similarity = (Size of Intersection) / (Size of Union)
```

Any pair of tweets with a similarity score above the 70% threshold is grouped together.

User Propagation Graph (DFS)

The `GraphAnalyzer.java` class finds toxic user clusters.

1. **Graph Building:** A `Map<String, User>` is built. When a retweet (e.g., `RT@userA`) is found, a directed edge is added from the

retweeter to `userA`.

2 **Cluster Finding:** The algorithm performs a DFS starting from every user who has posted a toxic tweet. The DFS explores the graph, and all users visited in a single search form a "cluster" of users connected to that toxic tweet. A `Set` of visited users ensures each user is only added to one cluster.

UI Design

The user interface is built using standard, pure JavaFX components with a clean and functional layout.

Layout: The root is a `BorderPane`, which provides a clean structure for the top controls, center table, and bottom results area. VBox and HBox containers are used for spacing and alignment.

Controls: Standard `Button` components trigger file loading and analysis. A `Label` provides the title, and a `TextArea` (set to non-editable) displays results.

`TableView`**:** This is the central UI component. It is bound to an `ObservableList<Tweet>`, so it updates automatically when the data changes.

Custom `CellFactory`**:** The "Tweet Text" column uses a custom cell factory. This allows for advanced rendering:

If a tweet is toxic, it uses a `TextFlow` node. This node can contain multiple `Text` objects, allowing us to style toxic words (bold, red) differently from the rest of the text.

If a tweet is not toxic, it uses a simple `Text` node with wrapping enabled, ensuring long tweets are fully readable.

Testing

This project does not currently include a formal unit testing suite (e.g., JUnit). Testing is performed manually by loading and analyzing the provided `tweets.csv` file.

Manual Test Cases:

Load `tweets.csv` and verify all tweets appear in the table.

Run analysis and confirm that tweets 102, 104, 105, and 106 are marked as toxic.

Confirm that the words "hate," "stupid," "dumb," "idiot," etc., are highlighted in red in the table.

Check the "Analysis Results" area for the "Similar/Duplicate Tweet Groups" report. Verify that tweets 102, 105, and 106 are grouped together.

Check the "Toxic User Propagation Clusters" report. Verify that `userB`, `userD`, `userE`, `userF`, and `userH` are identified in clusters.

Contributing

Contributions are welcome. If you would like to improve the tool, please follow these guidelines.

1. **Fork the Repository:** Start by forking the main project repository.

2. **Create a Branch:** Create a new branch for your feature or bugfix (e.g., `feature/add-junit-tests`).

3. **Make Changes:** Implement your changes and additions.

4. **Submit a Pull Request:** Push your branch to your fork and submit a pull request to the main repository, describing the changes you have made.

Reporting Bugs

Please report any bugs or issues by opening a new "Issue" in the project's GitHub repository. Provide a detailed description of the bug, the steps to reproduce it, and your environment (OS, Java version).

License & Acknowledgments

License

This project is licensed under the **MIT License**. A copy of the license is included in the repository. You are free to modify, distribute, and use this code for personal or commercial projects.

Acknowledgments

JavaFX: For the modern UI toolkit.

Lucide Icons: For the clean, open-source icons used in this documentation.